Generalizing Hardware Accelerator Optimizations for Directed ISA Compilation

Luke Herbelin, Joshua Gray, Sarkis Ter-Martirosyan
Advisors: Jonathan Balkind, Ziad Matni

December 1, 2025

1 Introduction

1.1 Research context

Processor design involves creating a processor that correctly performs desired functionalities while meeting Power, Performance, and Area (PPA) constraints. Designers first describe a processor's functionality at a high-level, often with an instruction set architecture (ISA). Then, they specify the processor's micro-architecture, which includes the construction of the internal processor and its essential components. This is represented formally at the Register Transfer Level (RTL) through a Hardware Description Language (HDL) such as Verilog or VHDL that describes the digital logic of the processor. RTL specification in languages like Verilog defines components such as wires, registers, and logic gates and connects these components to then model and verify a circuit. Finally, the designer can apply synthesis tools to transform complex circuits described in RTL into a technology-independent netlist consisting of generic logic gates such as AND, OR, and NOT. After realizing this layout and verifying correctness, designers finally fabricate a physical processor.

However, designing a processor often requires a complete understanding of how that processor is translated from its functionality and micro-architecture into its RTL specification. This is because RTL specification provides significant fine-grained control by allowing a hardware designer to work with the direct representation of the processor's implementation. This flexibility allows designers to implement micro-architectural optimizations, which perform behavior-preserving transformations on a micro-architecture while improving PPA metrics. However, this flexibility comes with the responsibility of manually maintaining each layer of abstraction in this process. For instance, after making small changes to the RTL specification, a designer may discover that it is actually necessary to revise processor functionality to meet PPA requirements, requiring edits to the ISA and micro-architecture. This complexity makes it difficult to construct and modify processor implementations which meet PPA requirements by requiring a holistic understanding of all aspects of hardware implementation ranging from inter-module control and data flow at RTL to a processor's micro-architecture and ISA.

To assist hardware designers in this process, High-Level Synthesis (HLS) tools were developed to automatically compile and optimize a circuit design from a behavioral hardware description written in a high-level programming language, such as C, to its RTL implementation. Generally, HLS performs optimizations as it compiles behavioral specifications to RTL, making HLS a widely-used tool in hardware design. HLS often relies on abstracting away specification details such as micro-architectural timing, making it especially powerful in the context of processors that contain many loops with the potential for

parallelization. Consequently, HLS is less effective for processors that require more specification, control, and hazard detection logic that is common with explicit fetch-execute cycles in instruction-based processors. Unfortunately, this means that HLS approaches exclude most instruction-based processors, making HLS practical primarily for non-instruction-based processors such as hardware accelerators.

1.2 Hardware Accelerators

With the increasing demand for high performance computing in specialized tasks such as matrix multiplication, hardware designers have turned to hardware accelerators [2]. These are processors with specialized micro-architectures designed to optimize performance and efficiency for specific tasks. Widely-used hardware accelerators include Field-Programmable Gate Arrays (FPGAs) used for the rapid ideation of hardware designs and Graphics Processing Units (GPUs) designed to drive powerful graphics and power AI systems. Additionally, another notable example includes Google's Tensor Processing Unit (TPU)[5], which uses specialized optimizations with systolic arrays to improve the efficiency of machine learning and artificial intelligence operations such as matrix multiplication and activation functions.

Because hardware accelerators are built with specialized micro-architectures, designers can develop new types of specialized optimizations that are specific to these hardware accelerators. These optimizations take advantage of their unique target micro-architectures to improve PPA metrics. The highly-specialized nature of hardware accelerators introduces a significant disconnect between optimizations designed for general-purpose processors and hardware accelerators.

We recognize that in both RTL and HLS, hardware designers struggle with effectively articulating optimizations. In RTL, large amounts of fine-grained control in the processor implementation makes it challenging to write and perform optimizations that work on different micro-architectures. Similarly, despite the fact that HLS can perform automatic optimizations at compile-time, HLS can only apply optimizations to primarily non-instruction-based processors. Because of these obstacles in describing and performing optimizations in both paradigms, it is difficult to study and understand the nature of a hardware accelerator's specialized optimizations, leaving applications of specialized optimizations to general-purpose instruction-based processors unexplored. Conversely, this also results in a difficulty in understanding how optimizations for general-purpose processors could be applied and fine-tuned for particular hardware accelerators.

1.3 Problem Statement

Without adapting specialized optimizations to general-purpose processors, hardware designers cannot employ the vast set of powerful optimizations specific to hardware accelerators on other processors. If these existing optimizations could be adapted into general variants that can be applied to other processors, hardware designers would gain a wide variety of new optimizations which they may apply at their own discretion. Similarly, applying optimizations for general-purpose processors to hardware accelerators can illuminate a better understanding of hardware accelerator micro-architectures and help designers devise general rules in targeting optimizations for hardware accelerators.

Furthermore, many optimizations for general-purpose processors are not area efficient or have a high area cost because they account for dynamic behavior, such as branch prediction. Hardware designers often do not want to pay this area cost and thus do not adapt these optimizations to hardware accelerators. As a result, the true cost and benefit of applying many optimizations from general processors to hardware accelerators is relatively unexplored.

This project seeks to further understand hardware accelerator's micro-architectures, their specialized op-

timizations, and how other generalized optimizations can be applied to them. We seek to determine if one can apply the optimizations that a hardware accelerator employs to general purpose processors, and other common optimizations to hardware accelerators. Analyzing this requires an understanding of instruction set architecture information. Unfortunately, traditional HDLs operate at a level that may not have this specification-level information. Meanwhile, although derived from behavioral specification, HLS approaches fail to differentiate which hardware components are available for modification and the set of invariants that must hold after performing hardware transformations. Recognizing these shortcomings in existing approaches, we concluded that a micro-architectural representation which encodes specification-level details is necessary to both generalize hardware accelerator optimizations and apply optimizations designed for general-purpose processors to hardware accelerators. We will use a framework called Isacomp and its Directed ISA Compilation functionality to maintain this micro-architectural specification to develop general algorithms that perform sufficiently well for multiple types of processors.

1.4 Isacomp

UCSB's ArchLab provides a solution that synthesizes both RTL and HLS paradigms into one framework: Isacomp. Isacomp borrows the automatic optimizations of high-level synthesis while maintaining the high degree of fine-grain control that RTL provides. Isacomp employs Directed ISA Compilation, which refers to manually applying optimizations to the graph representation of an ISA implementation. This process starts with users implementing an instruction set architecture and defining micro-architectural specifications with the Sail language, the ILA language, and Isacomp's custom HDL *isa*. After implementing the micro-architecture, Isacomp generates a directed resource graph that represents the state, behavior, and operations of that micro-architecture. From this, a user can select an optimization, such as pipelining, on a selected set of vertices and edges to perform on a resource graph. Furthermore, Isacomp can output a Verilog program that represents the optimized resource graph. Because Isacomp represents a micro-architecture with a resource graph and provides optimizations as black-box algorithms applied to resource graphs, hardware designers can visually understand how an optimization modifies a resource graph and how that translates to hardware without requiring an understanding of the exact implementation of that optimization. With these automatic, abstracted optimizations that the user can manually apply, Isacomp allows for a high degree of control over the implementation of a design in hardware while still maintaining the efficiency of automatic HLS tools, acting as a hybrid option between the RTL and HLS design paradigms.

2 Related Work

The idea of generalizing optimizations has been explored in the context of traditional optimizations. For example, traditional modulo based scheduling for pipelining is too rigid to be applied effectively to every architecture since it is unaware of specific micro-architectural constraints. As a solution, there exist frameworks that encapsulate a system of micro-architectural constraints in order to apply modulo-based scheduling effectively to specific architectures. [8] Since it is possible to modify general algorithms to better consider architectural constraints, we want to apply this approach to specialized optimizations for hardware accelerators.

Additionally, many powerful optimizations are designed to work with general components commonly found in different processors, despite having different micro-architectures. For example, re-timing is an optimization that focuses on relocating registers along latency-constrained paths to decrease the clock period of the circuit [7]. Because registers are common in most processors such as hardware accelerators, a designer can likely apply re-timing to other processors whose micro-architectures include registers.

Hence, framing optimizations over such general structures allows designers to port these optimizations to other designs which contain these same components and latency constraints. This reduces an optimization's dependency on a particular micro-architectural structure.

Many specialized processors and hardware accelerators employ a micro-architectural element known as a systolic array. Systolic arrays are a specialized parallel processing architecture or unit to avoid data or memory bottlenecks for high throughput operations like matrix multiplication. These arrays pass data through a grid of identical "processing elements" to reuse data as it is "pumped" through the system. This structure allows several operations to be run concurrently while sequentially obtaining inputs and outputs. Systolic arrays can be automatically generated on FPGAs to employ a variety of optimizations related to the micro-architecture, including latency hiding for pipeline stalls and SIMD vectorization by duplicating the compute units in processing elements [9]. This underscores the previously-mentioned dichotomy; these tools only function with FPGAs, yet systolic arrays and these optimizations can be applied to many other types of processors. We believe that with an extension of specialized optimizations, we can allow such improvements to be widely applicable to other processors.

An example of a hardware accelerator that has been transformed through optimizations as part of its design process is the Reduceron [6]. The Reduceron is a special-purpose processor for running lazy functional programs. It has a unique architecture that performs 6 semantic reduction rules in parallel in one clock-cycle and has no pipelining. The most recent design of the Reduceron sees significant performance increases through optimizations centered around performing dynamic analysis, which is common in optimizations for other processors like branch-prediction. Optimizations like branch-prediction are typically not seen in hardware accelerators because they can be cost-inefficient. Studying these specialized optimizations the designers made to the Reduceron could inform how these optimizations compare to common optimizations for instruction-based processors.

Existing work on hardware accelerators illustrates that hardware designers can develop an abstract representation of a hardware accelerator. One such formalism for this high-level abstraction is Instruction-Level Abstraction (ILA), which provides a specification style applicable to both general-purpose processors and accelerators for modeling and verification [4]. In an ILA model, a design's functionality is described as a set of instructions in the same way that one describes a processor architecture through an instruction set architecture (ISA). However, ILA extends ISA through a hierarchical structure that captures accelerator control/data-path behavior at different levels of abstraction, aiding in behavior specification and implementation. Additionally, ILA supports verification by checking refinement, which refers to the fact that an implementation conforms to an ILA specification, and by establishing ILA compatibility between design variants via equivalence checking, analogous to ISA compatibility in processors. Hence, existing methods to abstract and generalize hardware accelerators through ILA suggest that there may be a way to generalize accelerator-specific optimizations by expressing them as semantics-preserving transformations over a shared abstract model.

Furthermore, existing work connects ILA-based specifications to existing memory consistency models (MCMs) with a facet abstraction, where facets allow different agents to observe different versions of shared variables [3]. Using program sketches and MCM axioms, this approach generates Satisfiability Modulo Theories (SMT) constraints that can be solved to either produce a counterexample execution or establish correctness. Conceptually, executions can be interpreted as a graph where vertices represents events, edges encode ordering relations such as happens-before, and facets act as labels that distinguish which agent's view an event belongs to. This demonstrates that with a graph representation, one can represent memory consistency with ILA specification. Hence, existing work with ILA and MCM suggests that studying the correctness of and generalizing accelerator-specific optimizations will require a graph

representation of that processor's micro-architecture.

3 Proposed Solution

3.1 Overview

We assert that abstracting micro-architectures to be free of their specific micro-architectural context allows us to extend processor-specific optimizations to any processor that meets the general structural requirements of the optimization rather than targeting optimizations for a specific micro-architecture. Incorporating these generalized optimization algorithms in a directed-ISA compilation framework like Isacomp improves the capabilities of such frameworks to optimize hardware accelerators and other kinds of processors. Concretely, there are two directions of work this project will take. One is applying micro-architecture optimizations for general processors to hardware accelerators. The other is seeking to apply micro-architecture optimizations specific to hardware accelerators to general processors.

3.2 Developing Optimizations

Employing current research literature on hardware architectures and their associated specialized optimizations, we will implement a processor's instruction set architecture in the Isacomp framework to then generate a resource graph representation of that micro-architecture. Isacomp's intermediate resource graph representation offers an architectural-agnostic perspective of an instruction set architecture, effectively decoupling architecture-specific context from an optimization and extending it to be applicable to any processor. Because of this, we can apply these general-purpose optimizations to hardware accelerators despite their specialized micro-architectures.

The generalization of all hardware resources as `isa_states` further allows us to explore similarities between different micro-architectures before and after applying different types of optimizations. Through this exploration, we will generalize optimizations that were specifically designed for certain hardware accelerators so that they can run on other micro-architectural resource graphs.

Figure 1 illustrates an example of representing the MIPS add instruction as a resource graph. We observe input structures such as the program counter (`pc`), which serves the role of an input for each instruction. Additionally, we immediately recognize the MIPS add instruction's micro-architectural invariants: the presence of the data memory (`dmem`), instruction memory (`imem`), and register file (`rf`) structures. We also notice the intended output of adding the source register with the target register and storing the sum into the destination register. Hence, we can write formal optimization algorithms for the micro-architectural "shape" of these processors rather than for a particular micro-architecture. With this insight, we develop formal methods to articulate the generality of processor-specific optimizations and devise generalized optimizations applicable to other processors.

For example, one specialized optimization that we will generalize is state-machine pipelining, a specialized type of pipelining that involves instantiating copies of different states in order to inject multiple instructions when a state lasts multiple cycles. Successfully generalizing state-machine pipelining requires modifying this optimization to work on resource graphs and transforming micro-architectural preconditions to the more general resource-graph-specific conditions.

3.3 Implementing Accelerators

To successfully generalize specialized optimizations specific to hardware accelerators, we believe we must extend the current Isacomp compiler with new features needed to implement hardware accelera-

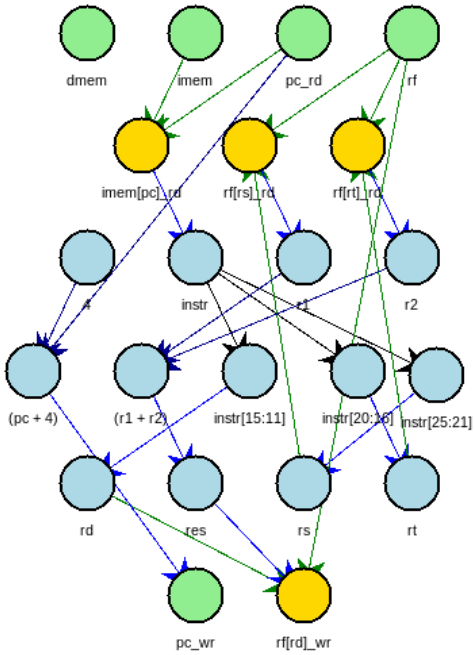

Figure 1: Resource Graph representation of the MIPS add instruction

tors. Drawing from our experiences in implementing hardware accelerators such as OpenTPU[1] in the Isacomp framework, we recognize that correctly expressing a rich set of hardware accelerators requires extending Isacomp's current static loop unrolling to include dynamic loop bounds and introducing type safety to aid in program correctness.

After implementing hardware accelerators with these new compiler features, we plan to compile the intermediate resource graph representation of a micro-architecture into Verilog. With this translation, we can directly study the micro-architectural structural and scheduling transformations at the register-transfer level after applying generalized algorithms. Hence, we will simulate this Verilog file to assess program correctness, measure the effectiveness of these optimizations, and explore potential tradeoffs that occur in applying an optimization.

4 Evaluation and Implementation Plan

In the following sections, we outline a general methodology to perform our research, methods to evaluate the effectiveness of generalized optimizations by measuring PPA metrics, and a detailed plan to implement these optimizations in the Isacomp framework.

4.1 General Methodology

We will begin our study by first selecting hardware accelerators and implementing them into Isacomp. After identifying 3 different hardware accelerators, we will then write descriptions of their instruction set architectures using one of the supported languages in Isacomp, such as SAIL or ILA. After these hardware accelerators have been implemented into the framework, Isacomp will generate the resource graph representation of the processors that the rest of our work will utilize. In order to ensure that we have correctly implemented the instruction set architectures, we will compile the generated resource graphs to Verilog and run simulations to verify correctness.

After we have an accurate resource graph representation of a hardware accelerator, we can apply the general-purpose optimizations included in Isacomp to those hardware accelerators. In this way, we will explore the effects of applying optimizations designed for general-purpose processors to hardware accelerators which are represented as a resource graph.

Furthermore, we need to identify specialized optimizations for hardware accelerators to extend to general-purpose processors. We will focus on optimizations that have potential benefits if implemented in a context apart from their specific micro-architecture. After obtaining a sufficient understanding from the optimization's documentation and publications, we will look to adapt the optimization such that it can be performed on a resource graph representation of a micro-architecture. Through this generalization, Isacomp can apply these optimizations to any processor or micro-architecture that meets the optimization's architectural requirements.

Finally, after applying either a specialized or general optimization, we will compile the optimized resource graph to Verilog and perform PPA analysis. We can benchmark this new PPA data with the metrics gathered prior to the optimization.

4.2 Evaluation Plan

In order to determine the effectiveness of our generalized optimizations, we will perform a Power, Performance, and Area (PPA) analysis on different processors after applying these generalized optimizations. Power is measured in Watts and refers to the overall energy efficiency of the design, accounting for the Dynamic Power consumed and the Static Power leaked. Performance refers to how fast the processor can execute instructions and is measured in the maximum instructions per cycle (IPC) $\times$ clock frequency that a processor can achieve. As a note, optimizations such as pipelining can be applied to a micro-architecture to result in a faster execution of instructions in cycles, but may slightly decrease the frequency. Overall, performance can be improved by optimizing for instructions per second or the execution time of an application. Area refers to the physical area in silicon that an implementation uses, measured in mm^2. In addition to being relevant to physical hardware constraints, area is also directly proportional to the fabrication costs of the chip. As Power and Performance are optimized, the area of the design may often increase, creating a tradeoff that must be considered when determining if an optimization is beneficial.

In order to perform PPA analysis on our optimized processors, we will compile Isacomp resource graphs into Verilog and simulate the Verilog code with tools such as Verilator or other professional Verilog simulation tools provided by the UC Santa Barbara ArchLab. These Verilog simulations can then be analyzed by numerous tools. The ArchLab has a license to Synopsis PrimePower, a commercial software that can automatically analyze the power consumption of designs. To measure performance, there exist open source tools such as OpenTimer or OpenROAD ASIC available for us to utilize. Area can be measured by open source tools such as Yosys or commercial software like Synposis Design Compiler.

Rather than optimizing for a specific metric to definitively classify an optimization as effective, we seek to understand the general impact of the optimization on PPA metrics, efficiency, and cost effectiveness of the processor using the original processor as a benchmark. After applying the optimization, we will analyze the improvements and tradeoffs in metrics to formulate an accurate understanding of which goals the optimization works best for. We will also consider how the optimization performs on different micro-architectures and if it improves a given metric with less tradeoffs for certain micro-architectures. In this way, we measure the performance of an optimization by its ability to optimize for a given metric, as determined by the hardware designer, while minimizing the tradeoff in the other metrics.

Since the result of an optimization depends on the vertices and edges selected as inputs, there are several

potential ways to apply an optimization to a given micro-architecture, potentially yielding varying PPA metrics. Since enumerating all valid choices of vertices and edges is computationally intractable, we will instead choose a set of input vertices and edges that a hardware designer would likely choose. Then, using a Verilog simulation software, we will measure PPA metrics across all of these possible optimization choices while filtering out potential outlier outputs that arise from unrealistic inputs. Hence, studying all "reasonable" optimization choices helps us build a more complete understanding of how an optimization impacts a micro-architecture's PPA metrics.

4.3 Timeline

Winter Break

- 4 weeks - No concrete goals, serves as time to possibly get ahead with some tasks or spend more time reviewing more literature.

Winter Quarter:

Week 1 - 3:

- Implement 3 hardware accelerator representations in Isacomp.
- Add Isacomp compiler features/fixes to support implementation of the processors.
- Test Isacomp Verilog output of the processors for verification.

Week 4 - 5:

- Determine if the optimizations can be extended to the hardware accelerators and if so how to implement them.
- Measure PPA with Verilog simulations to measure the effectiveness of these optimizations on hardware accelerators across a range of inputs.

Week 6 - 10:

- Implement specialized optimizations from our set of hardware accelerators and use unit-testing with other testing methods to ensure correctness.
- Generalize specialized optimizations to apply them to general processors.
- Measure PPA with Verilog simulations and how well these optimizations function on the general processors and hardware accelerators.

Spring Quarter:

Week 1-2:

- Finalize PPA results into final table.
- Analyze tradeoffs between different optimizations.
- Explore the differences in generalization of various optimizations and the most optimal design methodologies.
- Analyze potential limitations to the generalization of optimizations.

Week 3-4

- Complete any remaining tasks from previous weeks

- Construct final conclusions from research
- Understand the implications of our research in the broader context of Computer Architecture
- Determine if there are next steps for our research beyond ERSP

Week 5-10:

- Create summaries, diagrams, graphs, and visualizations for poster
- Construct poster to present findings
- Create and practice final presentation
- Present research

5 Acknowledgements

Initial thoughts and ideas, particularly in the Research Context section, were motivated by the *Compiling ISAs to Processors* paper by Professor Balkind and his team that is still in review.

6 Previous feedback

Parker Carlson and Professor Matni left the following feedback on our initial proposal draft

- The problem statement did well to accurately describe our specific problem, but a more specific description of the higher level problem was needed
- More timeline detail was needed for the Spring quarter
- Our evaluation section needed to define how an optimization is considered successful in context of it's PPA metrics
- Our proposal was missing a specific methodology that describes the series of steps, experiments, and benchmarks our research will use
- Our related works analysis was good, but needed more sources

Our proposal draft received the following grade:

- Context/Motivation: A
- Problem Statement: A-
- Literature Review: B
- Proposed Solution: A
- Technical details: A
- Evaluation/Timeline: A-
- Writing: A

References

[1] Andrew David Alex. Matrix multiplication is almost all you need. Master's thesis, UC Santa Barbara, 2024.

[2] Philip Canoza. Hardware accelerators and optimization algorithms for unconventional computing. Master's thesis, UC Berkely, January 13, 2023.

[3] Yatin A. Manerkar Aarti Gupta Margaret Martonosi Sharad Malik Hongce Zhang, Caroline Trippel. Ila-mcm: Integrating memory consistency models with instruction-level abstractions for heterogeneous system-on-chip verification. *ACM*, 2018.

[4] Bo-Yuan Huang, Hongce Zhang, Pramod Subramanyan, Yakir Vizel, Aarti Gupta, and Sharad Malik. Instruction-level abstraction (ila): A uniform specification for system-on-chip (soc) verification. *ACM Trans. Des. Autom. Electron. Syst.*, 2018.

[5] Norman P. Jouppi, Cliff Young, Nishant Patil, David Patterson, Gaurav Agrawal, Raminder Bajwa, Sarah Bates, Suresh Bhatia, Nan Boden, Al Borchers, Rick Boyle, Pierre luc Cantin, Clifford Chao, Chris Clark, Jeremy Coriell, Mike Daley, Matt Dau, Jeffrey Dean, Ben Gelb, Tara Vazir Ghaemmaghami, Rajendra Gottipati, William Gulland, Robert Hagmann, C. Richard Ho, Doug Hogberg, John Hu, Robert Hundt, Dan Hurt, Julian Ibarz, Aaron Jaffey, Alek Jaworski, Alexander Kaplan, Harshit Khaitan, Andy Koch, Naveen Kumar, Steve Lacy, James Laudon, James Law, Diemthu Le, Chris Leary, Zhuyuan Liu, Kyle Lucke, Alan Lundin, Gordon MacKean, Adriana Maggiore, Maire Mahony, Kieran Miller, Rahul Nagarajan, Ravi Narayanaswami, Ray Ni, Kathy Nix, Thomas Norrie, Mark Omernick, Narayana Penukonda, Andy Phelps, Jonathan Ross, Matt Ross, Amir Salek, Emad Samadiani, Chris Severn, Gregory Sizikov, Matthew Snelham, Jed Souter, Dan Steinberg, Andy Swing, Mercedes Tan, Gregory Thorson, Bo Tian, Horia Toma, Erick Tuttle, Vijay Vasudevan, Richard Walter, Walter Wang, Eric Wilcox, and Doe Hyun Yoon. In-datacenter performance analysis of a tensor processing unit. *ISCA*, 2017.

[6] Matthew Naylor and Colin Runciman. The reduceron reconfigured and re-evaluated. *Journal of Functional Programming*, 2012.

[7] Carl Ebeling Soha Hassoun. Architectural retiming: pipelining latency-constrained circuits. *Design Automation Conference*, 1996.

[8] Zhiru Zhang Steve Dai1. Improving scalability of exact modulo scheduling with specialized conflict-driven learning. *Design Automation Conference*, 2019.

[9] Jie Wang, Licheng Guo, and Jason Cong. Autosa: A polyhedral compiler for high-performance systolic arrays on fpga. *The 2021 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, 2021.